

Emulations of Photoshop Filters and Creative Experimentation

Corbett Brown

6/03/05

INTRODUCTION

The goal of this project was to actually learn how those neat filter effects in Photoshop work. In order to experiment with different algorithms to create the effect, I used Matlab as a programming environment to load pictures in and alter the pixels in a way to create an effect. Once an effect is truly understood, one can alter certain aspects to the algorithm to creatively design your own imaging effects. That is the second goal I had in this project.

FREQUENCY-DOMAIN FILTERING

Starting off, I worked with 2-D filtering of images by working in the frequency domain. One of the techniques to filtering data is to convert the image into the frequency-domain and multiply it by the transfer function of the filter (which is already in the frequency-domain). The first step is to convert the source image to the frequency domain by using the 2D FFT function in Matlab called `fft2`. Then, I created the desired frequency response by creating a matrix with ones corresponding to the frequencies passed and zeros corresponding to the filtered frequencies. Using a windowing function called `fwind1`, I created the actual frequency response after taking the FFT of the impulse response outputted from `fwind1`. Next, I multiplied the FFTed image and the frequency response of the filter. Finally, I took the inverse FFT of the result using `ifft2`. Figure 1 shows an example of a desired and actual frequency response using the hamming window. It also shows the original picture compared to the filtered picture using the above frequency response. Although this does filter the image, I found that it was not the easiest way to experiment with in order to design or emulate an image effect. Therefore, I went into the space domain and filtered using convolution.

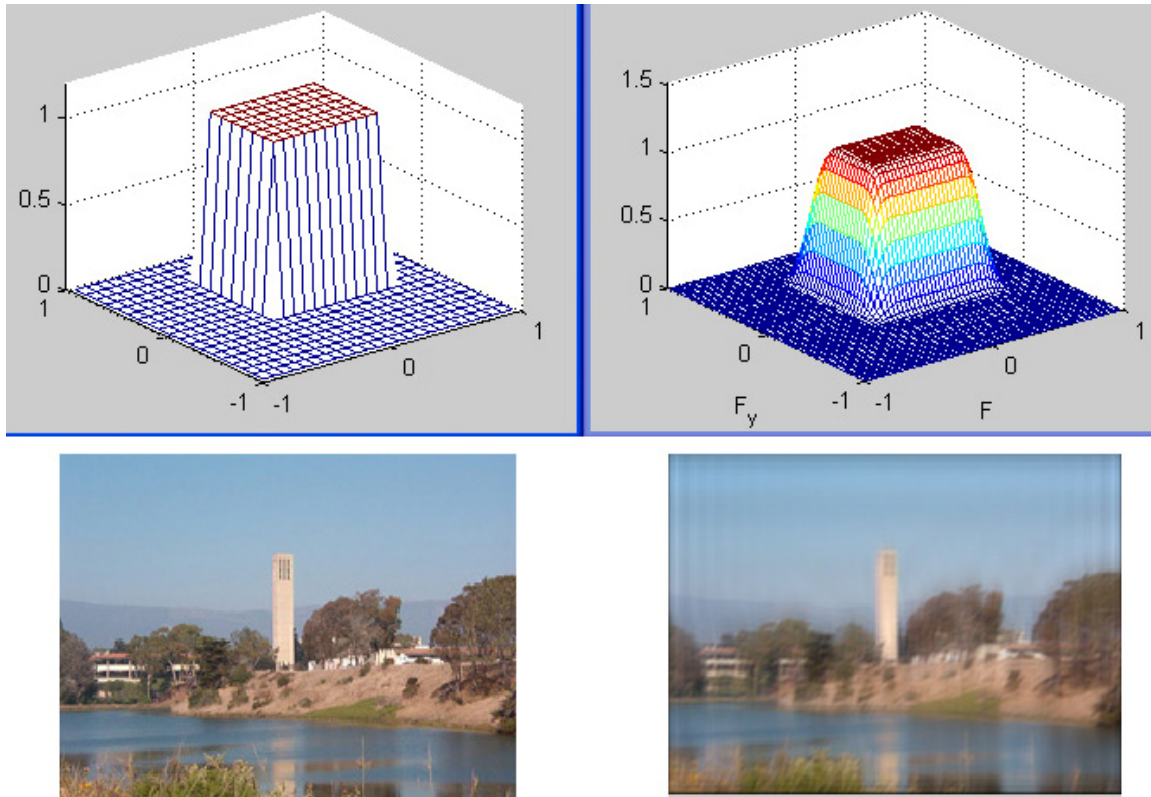


Figure 1: Upper left is desired frequency response. Upper right is actual frequency response. Bottom left is original image. Bottom right is filtered image.

CONVOLUTION FILTERING

A much more practical way of experimenting with different types of filters is to filter an image using convolution with a kernel (often called point spread function). The kernel is a much smaller matrix than the original image (common dimensions include 3 by 3 or 5 by 5). It is also commonly a square matrix, although it does not have to be. The way this filtering technique works is the kernel is placed on each pixel of the source image. The center of the kernel is lined up with the pixel, while the other numbers in the kernel are lined up with the neighboring pixels. Then, each pixel is multiplied by the number on the kernel for which it is lined up. They are added (and scaled down if necessary) to create the new pixel value that will replace the original pixel in the center. If convolution is taking place at the edges of the dimensions of the image, then some values in the kernel will not line up with a pixel. To fix this, the edges are usually padded with zeros or duplicated [1].

Sharpening Filter

In order to create more interesting effects, I started off by creating simpler ones. The first filter I made was a sharpening filter. In all my convolution filters, I used the Matlab function `imfilter`. The sharpening filter is a high-pass filter. In other words, it cuts down on the lower frequencies. To do this, the convolution must emphasize the differences the pixel values have with their neighbors. This is done by choosing a kernel where the center is positive and the neighboring pixels have either negative values or are zero (which means they have no influence). In order for the image to come out to be the same brightness, the kernel values must add up to one [1]. Here are a couple common sharpness kernels (using Matlab notation):

```
[0 -1 0;      [-1 -1 -1;
-1 5 -1;      -1 9 -1;
0 -1 0]       -1 -1 -1]
```

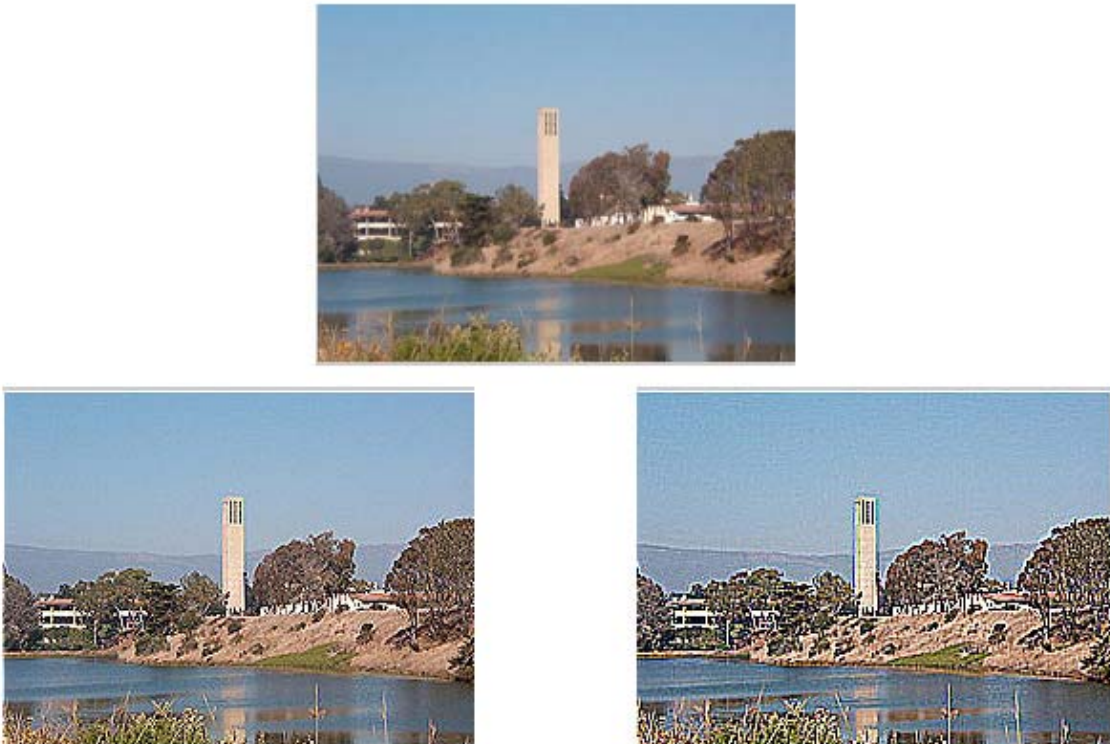


Figure 2: Top picture or original image. Bottom left is the image filtered with the left kernel above. Bottom right image is filtered with right kernel.

Edge Detection

In the sharpness filter, edge detection was used. Another way of creating a sharper image is to just create an image that just shows the edges and mix it with the original image.

Edge detection is also used to create other effects, as will be described later. The kernel for edge detection is similar to the sharpness filter, just decrease the center value by one.

So to turn the two sharpness kernels mention earlier into edge detection kernels, the center values will change to 4 and 8 respectively. After edge detection is performed, sometimes multiplying the image by a number to increase the brightness is necessary in order to better see the edges. Another difference between sharpness kernels and edge detection kernels is that the kernel values add up to zero [1]. Figure 3 shows an edge detected image multiplied by a scaling factor of 5 using the following kernel:

```
[-.125 -.125 -.125;  
-.125  1  -.125;  
-.125 -.125 -.125]
```

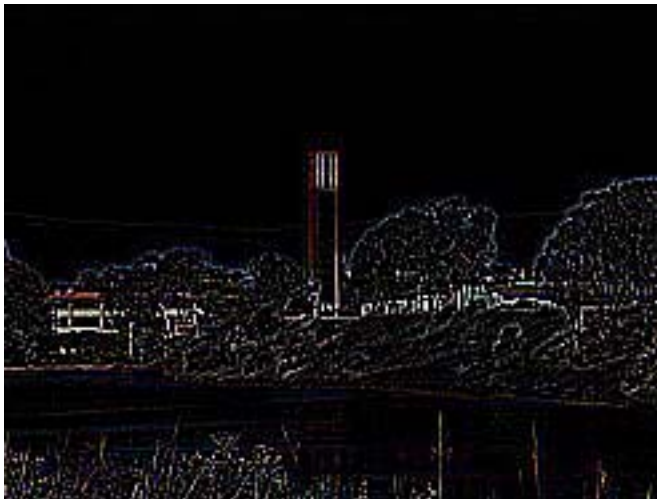


Figure 3: Edge Detection Filter

Emboss/Engrave

A picture can be made to look like it is embossed or engraved by using a certain type of edge detection filter that simulated light falling on the image in a certain direction. Light is scattered when it hits a rough surface, therefore a kernel is used that simulated this scattering by making parts of the edges in the image darker, and other parts (depending

on the direction of the light) brighter. The kernels used are below. Notice how the values are simply rotated to change the direction of the light. Figure 4 shows the four diagonal directions of light in the embossed images. An offset of 0.5 (the pixels are normalized to between 0 and 1) is added to see the full emboss/engrave because some values will become negative when applying the kernel. Matlab automatically assigns negative values to zero when displaying images, thus losing information. The upper two images look embossed and the lower two images look engraved [1].

[2 0 0;	[0 0 2;	[-1 0 0;	[0 0 -1;
0 -1 0;	0 -1 0;	0 -1 0;	0 -1 0;
0 0 -1]	-1 0 0]	0 0 2]	2 0 0]

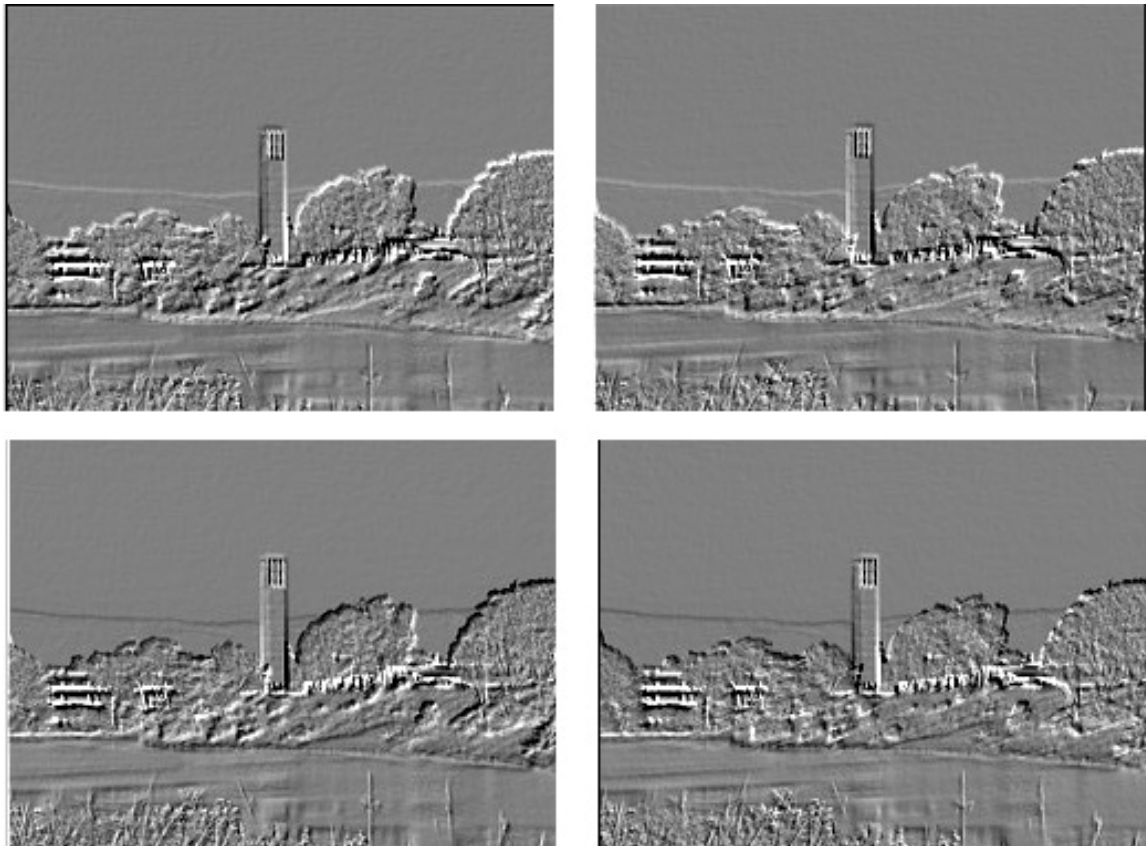


Figure 4: Upper left image uses farthest left kernel. Upper right image uses middle left kernel. Bottom left image uses middle right kernel. Bottom right image uses farthest right kernel.

Although the filtered image was offset by 0.5, it can also be offset by different values of red, green, and blue to create an embossed image with a different color. This can be

extended to the concept of bump mapping, which embosses an image onto a different image called a texture. A simple implementation of bump mapping just takes the previous embossed filter, but instead of offsetting by 0.5, the filtered image is added to the texture. Figure 5 shows different textures that are bump mapped [2].

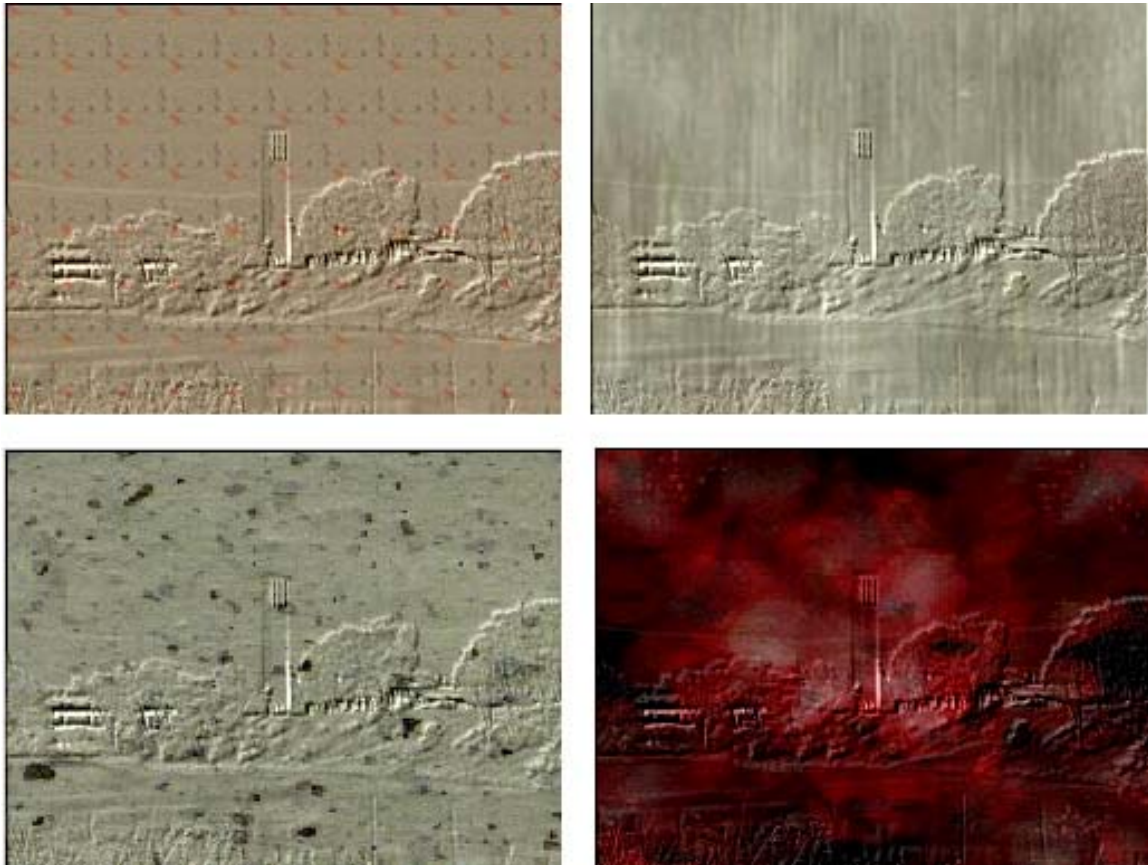


Figure 5: Bump Mapped textures.

This form of combining two images is often times a more interesting effect than just simply mixing two images together by taking the average of the two pixels from each image. Figure 6 shows a simple mixing of the bottom right texture from figure 5 [1].



Figure 6: Image mixing.

Blurring

On the other side of the spectrum, applying a low-pass filter makes an image look blurry. The blurring kernel characteristically has all positive values in the matrix. The values in the matrix are sometimes all between 0 and 1 so that they all add up to one. If there are some greater than one, then the final image should be scaled down to equal one in order to have the same brightness as the original image. The kernels are set up this way in order to average out the pixels. This makes it so that there are fewer differences between neighboring pixels, thus making the edges less pronounced. A simple implementation of a blurring kernel is a simple average of all the pixels (including the center) that are in the neighborhood defined by kernel dimensions [1]. Here is a common averaging kernel:

```
[1/9 1/9 1/9;  
 1/9 1/9 1/9;  
 1/9 1/9 1/9].
```

Another popular kernel creates what is called a Gaussian blur. The special thing about the Gaussian kernel is that it can be decomposed into a row vector and a column vector that can simply be multiplied together to create the kernel. The column vector is a transposed version of the row vector. This can save computation time (on the order of 6 times). The row vector is first convolved with the image to create an intermediate image. Then the intermediate image is convolved with the column vector to create the final image with the Gaussian blur. The following shows the vector that can be used to create a Gaussian blur of order 11: [1 11 55 165 330 462 462 330 165 55 11 1]. Figure 7 shows the averaging filter and the Gaussian blur with order of 11 [1].



Figure 7: Top picture is original image. Bottom left picture is image with average filter. Bottom right picture is image with Gaussian blur order 11 filter.

Glowing Edges

One of the cool filtering effects from Photoshop is the glowing edges effect, commonly found in the stylize section of the Filter menu. The glowing edges are created by combining the blurring effect with edge detection. If you noticed in the edge detected image in Figure 3, the edges are multiple colors due to some of the RGB values being zeroed out (or close to zeroed out). This image can be extended to the glowing edge effect by making just the edges brighter. This is done by using a threshold that says if any of the RGB values are greater than the threshold, they are multiplied by the scaling factor to make them brighter. Otherwise, if it is below the threshold the pixel is left alone. With pixels normalized to 0-1, I experimented and found that 0.02 was a good threshold value. The threshold determines how many edges you see. Also, to make the edges thicker, a Gaussian blur filter is applied to the image before the edge detection filter is used. This spreads out the edges making it thicker and more suitable for a glowing look. Figure 8

shows the glow edge effect applied using Matlab and it also shows the effect when using Photoshop for comparing purposes.

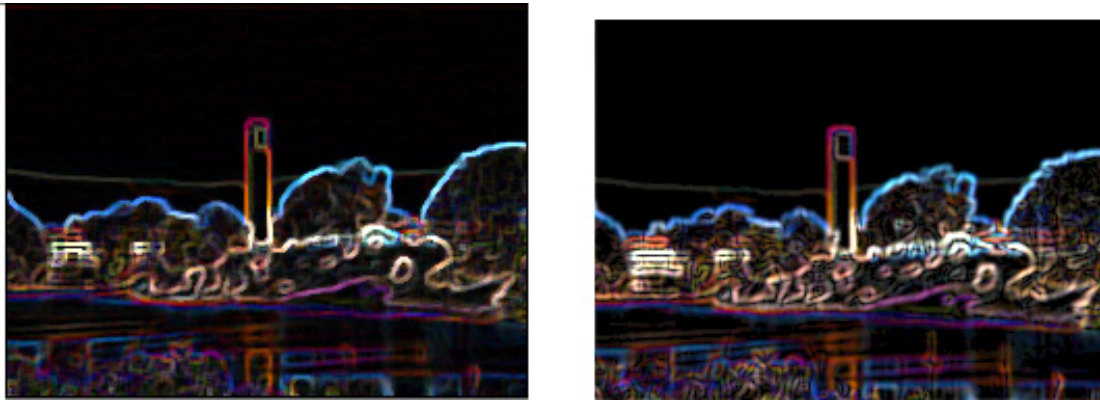


Figure 8: Glowing Edges effect. Left picture is from Adobe Photoshop. Right picture is from algorithm used in Matlab.

PIXEL MANIPULATION

Many Photoshop effects can be created by going through pixel by pixel and doing some sort of manipulation that is not convolution.

Solarize

The solarize filter in Photoshop uses a simple algorithm. This effect is a special type of inversion. Simply inverting the image is done by taking each pixel and subtracting each RGB value by the highest value (in my case is 1). This creates the opposite color of the original RGB color. Using a different value other than 1 and then scaling the final result can create other interesting colorizing effects. I also tried scaling by the original pixel value, which makes kind of a glowing effect. Figure 9 shows inversion and different creative experiments with the algorithm. Solarizing is a simple modification of inversion. A threshold of half the maximum brightness value (in my case is 0.5) is used to make red, green, and blue values of the pixels above the threshold inverted, while the pixels below the threshold are left alone. In other words, only the bright values of the red, green, or blue components are inverted. This makes the image overall darker. An interesting modification of solarizing can be done by changing the threshold or swapping the inversion to affect the darker values instead of the brighter ones. This is also shown in

Figure 9. Figure 10 shows the solarizing effect in Matlab and its comparison to the Photoshop version [3].



Figure 9: Upper left is completely inverted image (where subtraction is from one). Upper right image is inversion with subtraction from 0.75. Bottom left image is inversion with subtraction from 0.5 and scaling with original pixel value. Bottom right image is the opposite of solarizing.

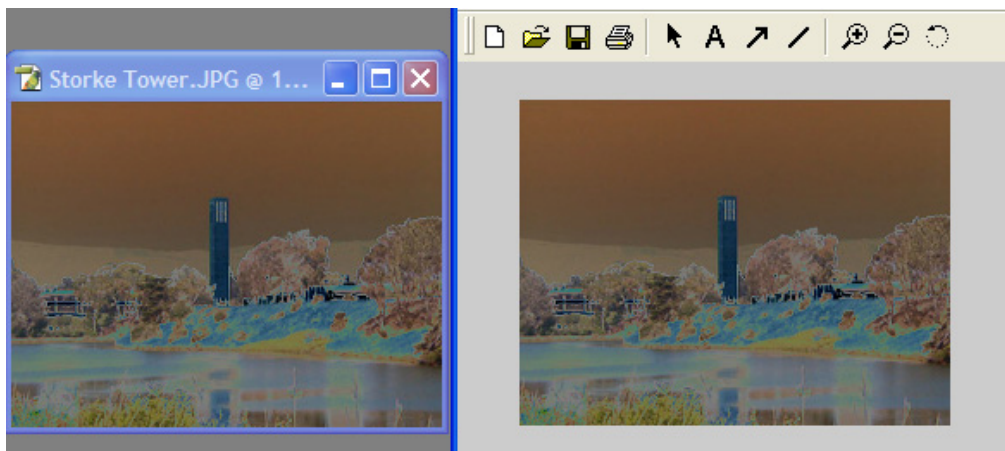


Figure 10: Solarize. Left image is from Photoshop. Right image is form Matlab.

Pixelate

Pixelation is an effect that makes the pixels in the image a larger area. The pixels can change to a certain shape to create a crystallizing effect. I simply implemented the effect leaving the shapes of the pixels intact, and just increasing the square area. A parameter called the pixel width can often be set by the user to determine how big the pixels will become. I used a pixel width of 10. The algorithm goes through every 10th pixel horizontally and vertically and makes every pixel in the 10 by 10 square area the same value. The value used can either be just one pixel in the square area, or an average of all of those pixels. My algorithm uses the average. Figure 11 shows the result [4].



Figure 11: Left is original image. Right is pixelated image.

Posterizing

I have not found a complete replication of the Poster Edges effect found in Photoshop, but I have made some progress. The idea behind posterizing is that the pixel values are quantized to a certain number of levels. My first implementation quantizes the red, green, and blue values to equally spaced values that transverse the pixel range. This creates the posterizing effect, but the colors significantly change. I also tried an implementation that detects the dominant red, green, and blue values separately and quantizes the RGB values to those values. The colors are also significantly changed because red, green, and blue values are treated separately. Finally, I tried an implementation that detects the dominant RGB value combined and quantized the pixels to those levels. Figure 12 shows my three implementations [5].



Figure 12: Posterizing. Upper left is original image. Upper right is quantization with equally spaced values. Bottom left is quantization with dominant RGB values separated. Bottom right is quantization with RGB values combined.

CONCLUSION

Filtering is a powerful effect that can do drastic changes to images. Understanding the algorithms themselves allow one to be more creative in image manipulation, and also allows one to understand the parameters associated with filters/effects found in such programs as Adobe Photoshop.

REFERENCES

- [1] "An Introduction to Digital Image Processing,"
<http://www.gamedev.net/reference/articles/article2007.asp>.
- [2] "Secrets of Simple Image Filtering,"
<http://www.gamedev.net/reference/articles/article1994.asp>
- [3] "Jerry's Java Image Processing Pages," <http://www.jhllabs.com/ip/index.html>.
- [4] "Image Processing for Dummies with C# and GDI+ Part 5,"
<http://www.codeproject.com/cs/media/displacementfilters.asp>.
- [5] "AxiomX – Posterize Algorithm," <http://www.axiomx.com/posterize.htm>.

MATLAB CODE

Here is the Matlab code I used to emulate and experiment with effects:

```
function [filteredimage] = phoshofilt(orimage, type, option, texture)
figure, imshow(orimage);
orimage = im2double(orimage);
%In this project, I investigated different filter/effects found in
%Photoshop. Starting off, I looked at basic filters that can easily be
%implemented using convolution of small kernels(also called Point Spread
%Function.
if length(type) == 7 & type == 'sharpen'
    if option == 1
        kern = [0 -1 0;
                -1 5 -1;
                0 -1 0];
    else
        kern = [-1 -1 -1;
                -1 9 -1;
                -1 -1 -1]; %sharpness filter
    end;
    fimage = imfilter(orimage, kern);

elseif length(type) == 7 & type == 'edgedet'
    kern = [-.125 -.125 -.125;
            -.125 1 -.125;
            -.125 -.125 -.125]; %edge detection
    fimage = imfilter(orimage, kern);
    fimage = fimage * 5;

elseif length(type) == 6 & type == 'emboss'
    if option == 1 %different options have different directions of the light
        kern = [2 0 0;
                0 -1 0;
                0 0 -1];
        %kern = [-0.25 0 0 0 0;
                0 0 0 0 0;
                0 0 0 0 0;
                0 0 0 0 0;
                0 0 0 0 0];
    end;
    fimage = imfilter(orimage, kern);
end;
```

```

%    0 -0.5 0 0 0;
%    0  0  1 0 0;
%    0  0  0 0 0;
%    0  0  0 0 0];
else
    kern = [0 0 2;
            0 -1 0;
            -1 0 0];
    %kern = [0 0 0 0 -0.25;
            %    0 0 0 -0.5  0;
            %    0 0 1  0   0;
            %    0 0 0  0   0;
            %    0 0 0  0   0];
end;
orimage = rgb2gray(orimage);
fimage = imfilter(orimage,kern);
fimage = fimage + 0.5;

elseif length(type) == 7 & type == 'engrave'
    if option == 1
        kern = [-1 0 0;
                0 -1 0;
                0  0 2];%engrave
        %kern = [0 0 0 0  0;
                %    0 0 0  0  0;
                %    0 0 1  0  0;
                %    0 0 0 -0.5 0;
                %    0 0 0  0 -0.25];
    else
        kern = [0 0 -1;
                0 -1 0;
                2 0  0];%engrave
        %kern = [0  0  0 0 0;
                %    0  0  0 0 0;
                %    0  0  1 0 0;
                %    0 -0.5 0 0 0;
                %   -0.25 0  0 0 0];
    end;
    orimage = rgb2gray(orimage);
    fimage = imfilter(orimage,kern);
    fimage = fimage + 0.5;

elseif length(type) == 10 & type == 'bumpmapped'
    kern = [2 0  0;
            0 -1 0;
            0 0 -1];
    orimage = rgb2gray(orimage);
    emimage = imfilter(orimage,kern)/option; %option scales down the embossment
    figure, imshow(texture);
    texture = im2double(texture);
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            for K = 1:3,
                fimage(I,J,K) = emimage(I,J) + texture(I,J,K);
            end;
        end;
    end;
end;

```



```

end;

elseif length(type) == 3 & type == 'mix'
    figure, imshow(texture);
    texture = im2double(texture);
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2), %option determines ration of first image to second
            for K = 1:3, %should be number between 0 and 1
                fimage(I,J,K) = option*orimage(I,J,K) + (1-option)*texture(I,J,K);
            end;
        end;
    end;

elseif length(type) == 7 & type == 'average'
    kern = [1/9 1/9 1/9;
            1/9 1/9 1/9;
            1/9 1/9 1/9]; %edge detection
    fimage = imfilter(orimage,kern);

elseif length(type) == 10 & type == 'gaussblur2'
    kern = [1 2 1;
            2 4 2;
            1 2 1]; %gaussian blur order 2
    fimage = imfilter(orimage,kern);
    fimage = fimage / 16;

elseif length(type) == 10 & type == 'gaussblur8'
    vert = [1; 8; 28; 56; 70; 56; 35; 8; 1]; %gaussian blur order 8
    hori = [1 8 28 56 70 56 35 8 1];
    kern = vert * hori;
    fimage = imfilter(orimage,kern);
    fimage = fimage / 65536;

elseif length(type) == 11 & type == 'gaussblur11'
    vert = [1; 11; 55; 165; 330; 462; 462; 330; 165; 55; 11; 1]; %gaussian blur order 11
    tempimage = imfilter(orimage,vert); %intermediate image with vertical kernel
    hori = [1 11 55 165 330 462 462 330 165 55 11 1];
    fimage = imfilter(tempimage,hori); %finale image
    fimage = fimage / 4194304;

elseif length(type) == 8 & type == 'glowedge'
    vert = [1; 4; 6; 4; 1]; %gaussian blur order 4
    hori = [1 4 6 4 1];
    kern = vert * hori;
    timage = imfilter(orimage,kern);
    timage = timage / 256;
    kern1 = [-1 0 1;
             -1 0 1;
             -1 0 1];
    ledgeimage = imfilter(timage,kern1);
    ledgeimage = im2uint8(ledgeimage);
    ledgeimage = im2double(ledgeimage);
    kern2 = [1 0 -1;
             1 0 -1;
             1 0 -1];
    redgeimage = imfilter(timage,kern2);

```

```

redgeimage = im2uint8(redgeimage);
redgeimage = im2double(redgeimage);
kern3 = [1 1 1;
         0 0 0;
        -1 -1 -1];
dedgeimage = imfilter(timage,kern3);
dedgeimage = im2uint8(dedgeimage);
dedgeimage = im2double(dedgeimage);
kern4 = [-1 -1 -1;
         0 0 0;
         1 1 1];
uedgeimage = imfilter(timage,kern4);
uedgeimage = im2uint8(uedgeimage);
uedgeimage = im2double(uedgeimage);
fimage = (ledgeimage + redgeimage + dedgeimage + uedgeimage)/4;
for I = 1:size(orimage,1),
    for J = 1:size(orimage,2),
        if fimage(I,J,1) > 0.02 | fimage(I,J,2) > 0.02 | fimage(I,J,3) > 0.02
            fimage(I,J,:) = fimage(I,J,:) * option; %option controls the brightness
        end;
    end;
end;

elseif length(type) == 6 & type == 'invert'
    fimage = abs(option - orimage)/option; %option determines what point to invert
    %option best used if between 0.5 and 1 to get different results

elseif length(type) == 8 & type == 'solarize'
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            for K = 1:3,
                if orimage(I,J,K) > 0.5
                    fimage(I,J,K) = 1 - orimage(I,J,K);
                else
                    fimage(I,J,K) = orimage(I,J,K);
                end;
            end;
        end;
    end;

elseif length(type) == 12 & type == 'brightinvert'
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            for K = 1:3,
                if orimage(I,J,K) < 0.5
                    fimage(I,J,K) = 1 - orimage(I,J,K);
                else
                    fimage(I,J,K) = orimage(I,J,K);
                end;
            end;
        end;
    end;

elseif length(type) == 10 & type == 'invertglow'
    fimage = abs(option - orimage)./orimage;

```

```

elseif length(type) == 8 & type == 'pixelate' %option is pixel width
    fimage(:, :, :) = zeros(size(orimage,1),size(orimage,2),3);
    for I = 1:option:size(orimage,1),
        for J = 1:option:size(orimage,2),
            for K = 1:3,
                for N = 0:option-1,
                    for M = 0:option-1,
                        for P = 0:option-1,
                            for Q = 0:option-1,
                                if I+N > 0 & I+N <= size(orimage,1) & J+M > 0 & J+M <= size(orimage,2) & I+P > 0
                                    & I+P <= size(orimage,1) & J+Q > 0 & J+Q <= size(orimage,2)
                                        fimage(I+N,J+M,K) = fimage(I+N,J+M,K) + orimage(I+P,J+Q,K);
                                    end;
                                end; %square pixelate
                            end;
                        if I+N > 0 & I+N <= size(orimage,1) & J+M > 0 & J+M <= size(orimage,2)
                            fimage(I+N,J+M,K) = fimage(I+N,J+M,K) / (option * option);
                        end;
                    end;
                end;
            end;
        end;
    end;
end;

```

```

elseif length(type) == 10 & type == 'resochange'
    %kern = [0 -1 0;
    %      -1 5 -1;
    %      0 -1 0]; %sharpness filter
    %temp2image = imfilter(orimage,kern);
    vert = [1; 11; 55; 165; 330; 462; 462; 330; 165; 55; 11; 1]; %gaussian blur order 11
    hori = [1 11 55 165 330 462 462 330 165 55 11 1];
    kern = vert * hori;
    tempimage = imfilter(orimage,kern);
    tempimage = tempimage / 4194304;
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            for K = 1:3,
                fimage(I,J,K) = tempimage(I,J,K) * option; %option determines how many to quantize
                fimage(I,J,K) = floor(fimage(I,J,K));
                fimage(I,J,K) = fimage(I,J,K) / option;
            end;
        end;
    end;
end;

```

```

elseif length(type) == 9 & type == 'posterize'
    %kern = [0 -1 0;
    %      -1 5 -1;
    %      0 -1 0]; %sharpness filter
    %temp2image = imfilter(orimage,kern);
    vert = [1; 11; 55; 165; 330; 462; 462; 330; 165; 55; 11; 1]; %gaussian blur order 11
    hori = [1 11 55 165 330 462 462 330 165 55 11 1];
    kern = vert * hori;
    tempimage = imfilter(orimage,kern);
    tempimage = tempimage / 4194304;
    redhistmat = histc(tempimage(:, :, 1)*255,0:255);
    redhistarr = zeros(256,1);

```

```

for N = 1:size(orimage,2), redhistarr = redhistarr + redhistmat(:,N); end;
[Y,reddomarr] = sortrows(redhistarr);
temp = zeros(option+1,1);
temp(1) = 900;
cnt = 256;
flag = 0;
for M = 2:size(temp),
    flag = 0;
    while flag == 0,
        for Q = 1:M-1,
            if reddomarr(cnt) < temp(Q)+((256/option)/1) & reddomarr(cnt) > temp(Q)-((256/option)/1)
                flag = 0;
            else
                flag = 1;
            end;
        end;
        cnt = cnt - 1;
    end;
    temp(M) = reddomarr(cnt);
end;
redvalues = temp(2:size(temp));
redvalues = redvalues / 255

greenhistmat = histc(orimage(:,:,2)*255,0:255);
greenhistarr = zeros(256,1);
for N = 1:size(orimage,2), greenhistarr = greenhistarr + greenhistmat(:,N); end;
[Y,greendomarr] = sortrows(greenhistarr);
temp = zeros(option+1,1);
temp(1) = 900;
cnt = 256;
flag = 0;
for M = 2:size(temp),
    flag = 0;
    while flag == 0,
        for Q = 1:M-1,
            if greendomarr(cnt) < temp(Q)+((256/option)/1) & greendomarr(cnt) > temp(Q)-((256/option)/1)
                flag = 0;
            else
                flag = 1;
            end;
        end;
        cnt = cnt - 1;
    end;
    temp(M) = greendomarr(cnt);
end;
greenvalues = temp(2:size(temp));
greenvalues = greenvalues / 255

bluehistmat = histc(orimage(:,:,3)*255,0:255);
bluehistarr = zeros(256,1);
for N = 1:size(orimage,2), bluehistarr = bluehistmat(:,N); end;
[Y,bluedomarr] = sortrows(bluehistarr);
temp = zeros(option+1,1);
temp(1) = 900;
cnt = 256;
flag = 0;

```

```

for M = 2:size(temp),
    flag = 0;
    while flag == 0,
        for Q = 1:M-1,
            if bluedomarr(cnt) < temp(Q)+((256/option)/1) & bluedomarr(cnt) > temp(Q)-((256/option)/1)
                flag = 0;
            else
                flag = 1;
            end;
        end;
        cnt = cnt - 1;
    end;
    temp(M) = bluedomarr(cnt);
end;
bluevalues = temp(2:size(temp));
bluevalues = bluevalues / 255
%redvalues = [0.1; 0.5; 0.6; 0.9; 0.95];
%greenvalues = [0.1; 0.15; 0.26; 0.39; 0.75];
%bluevalues = [0.13; 0.43; 0.53; 0.67; 0.79];

fimage = zeros(size(orimage,1), size(orimage,2),size(orimage,3));
for I = 1:size(orimage,1),
    for J = 1:size(orimage,2),
        for K = 1:size(redvalues),
            temp2(K) = abs(orimage(I,J,1) - redvalues(K));
        end;
        [dummy temp3] = min(temp2);
        fimage(I,J,1) = redvalues(temp3);
        for K = 1:size(greenvalues),
            temp2(K) = abs(orimage(I,J,2) - greenvalues(K));
        end;
        [dummy temp3] = min(temp2);
        fimage(I,J,2) = greenvalues(temp3);
        for K = 1:size(bluevalues),
            temp2(K) = abs(orimage(I,J,3) - bluevalues(K));
        end;
        [dummy temp3] = min(temp2);
        fimage(I,J,3) = bluevalues(temp3);
    end;
end;

elseif length(type) == 10 & type == 'posterize2'
%kern = [0 -1 0;
%    -1 5 -1;
%    0 -1 0]; %sharpness filter
%temp2image = imfilter(orimage,kern);
vert = [1; 8; 28; 56; 70; 56; 35; 8; 1]; %gaussian blur order 8
hori = [1 8 28 56 70 56 35 8 1];
vert = [1; 11; 55; 165; 330; 462; 462; 330; 165; 55; 11; 1]; %gaussian blur order 11
hori = [1 11 55 165 330 462 462 330 165 55 11 1];
kern = vert * hori;
tempimage = imfilter(orimage,kern);
tempimage = tempimage / 4194304;%65536;
tempimage = im2uint8(tempimage);
[indimage map] = rgb2ind(tempimage,65536);
indimage = double(indimage);

```

```

histmat = histc(indimage(:,0:65535));
histarr = zeros(65536,1);
for N = 1:size(orimage,2), histarr = histarr + histmat(:,N); end;
[Y,domarr] = sortrows(histarr);
temp = zeros(option+1,1);
temp(1) = 70000;
cnt = 65536;
flag = 0;
for M = 2:size(temp),
    flag = 0;
    while flag == 0,
        for Q = 1:M-1,
            if domarr(cnt) < temp(Q)+50 & domarr(cnt) > temp(Q)-50
                flag = 0;
            else
                flag = 1;
            end;
        end;
        cnt = cnt - 1;
    end;
    temp(M) = domarr(cnt);
end;
values = temp(2:size(temp));
%values = [0.3; 0.4; 0.5; 0.6; 0.7];
for L = 1:size(values),
    rgbvalues(L,:) = map(L,:);
end;
findimage = zeros(size(orimage,1), size(orimage,2));
for I = 1:size(orimage,1),
    for J = 1:size(orimage,2),
        for K = 1:size(rgbvalues),
            temp2(K) = 0;
            for R = 1:3,
                temp2(K) = temp2(K) + abs(orimage(I,J,R) - rgbvalues(K,R));
            end;
        end;
        [dummy temp3] = min(temp2);
        temp2image(I,J,:) = rgbvalues(temp3,:);
    end;
end;
kern = [1 1;
        1 1]; %gaussian blur order 1
fimage = imfilter(temp2image,kern);
fimage = fimage / 4;

elseif length(type) == 9 & type == 'thickedged'
    kern = [0 -1 0;
            -1 4 -1;
            0 -1 0]; %edge detection
    edgeimage = imfilter(orimage,kern);
    %kern = [3/32 0 -3/32;
            % 10 0 -10;
            % 3/32 0 -3/32]; %edge detection
    %edgeimage2 = imfilter(orimage,kern);
    %fimage = ((edgeimage.^ 2) + (edgeimage2.^ 2)).^ 0.5;
    %fimage(:,,:) = orimage(:,,:);

```



```

fimage(:,:,:) = zeros(size(orimage,1),size(orimage,2),3);
for I = 1:size(orimage,1),
    for J = 1:size(orimage,2),
        if edgeimage(I,J,1) > 0.1 | edgeimage(I,J,2) > 0.1 | edgeimage(I,J,3) > 0.1
            for N = -1:1:1,
                for M = -1:1:1,
                    if (I+N) > 0 & (I+N) < size(orimage,1) & (J+M) > 0 & (J+M) < size(orimage,2)
                        fimage(I+N,J+M,:) = [1 1 1];
                    end;
                end;
            end;
        end;
    end;
end;

elseif length(type) == 11 & type == 'experiment1'
    kern = [-1 0 1; 0 1 0; 1 0 -1];
    fimage = imfilter(orimage,kern);

elseif length(type) == 11 & type == 'experiment2'
    kern = [-1 -1 -1; 0 0 0; 1 1 1];
    fimage = imfilter(orimage,kern) * 4;

elseif length(type) == 6 & type == 'random'
    kern = [-.125 -.125 -.125; -.125 1 -.125; -.125 -.125 -.125]; %edge detection
    tempimage = imfilter(orimage,kern);
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            if tempimage(I,J,1) > 0.05 | tempimage(I,J,2) > 0.05 | tempimage(I,J,3) > 0.05
                temp2image(I,J,:) = [0 0 0];
            else
                temp2image(I,J,:) = [1 1 1];
            end;
        end;
    end;
    vert = [1; 8; 28; 56; 70; 56; 35; 8; 1]; %gaussian blur order 8
    hori = [1 8 28 56 70 56 35 8 1];
    kern = vert * hori;
    temp4image = imfilter(orimage,kern);
    temp4image = temp4image/65336;
    for I = 1:size(orimage,1),
        for J = 1:size(orimage,2),
            if temp2image(I,J,1) == 0
                temp3image(I,J,:) = temp2image(I,J,:);
            else
                temp3image(I,J,:) = temp4image(I,J,:);
            end;
        end;
    end;
    kern = [1 1; 1 1];
    fimage = imfilter(temp3image,kern);
    fimage = fimage / 4;
end; fimage = im2uint8(fimage); filteredimage = fimage; figure, imshow(fimage)

```